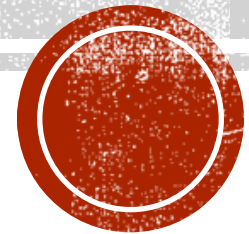


LECTURE 07

ListViews



Presentation Topics:

- In this presentation, we will learn the following topics:
 - ❑ Create a ListView
 - ❑ How to add items at runtime to a ListView
 - ❑ How to customize ListView items
 - ❑ How to display Images from a remote server

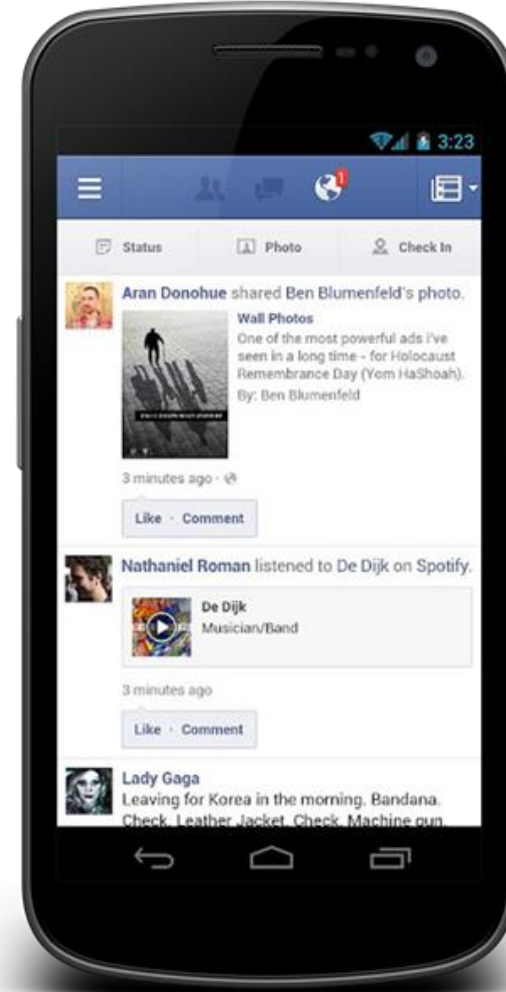
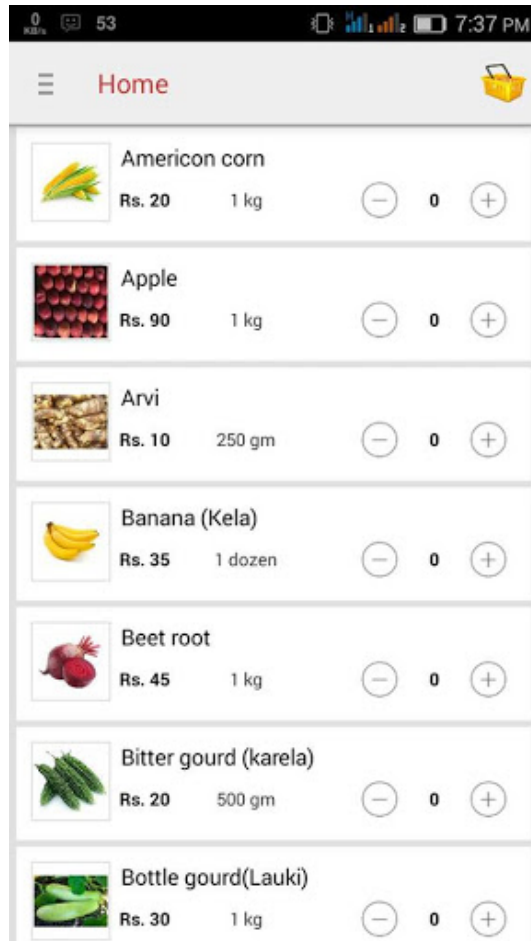


WHAT ARE LISTVIEWS:

- ❑ ListViews play a very important role in modern applications
- ❑ The posts you see in applications such as Facebook, are items in a ListView
- ❑ When you use an e-commerce application that displays a list of available products, most likely you are using a ListView

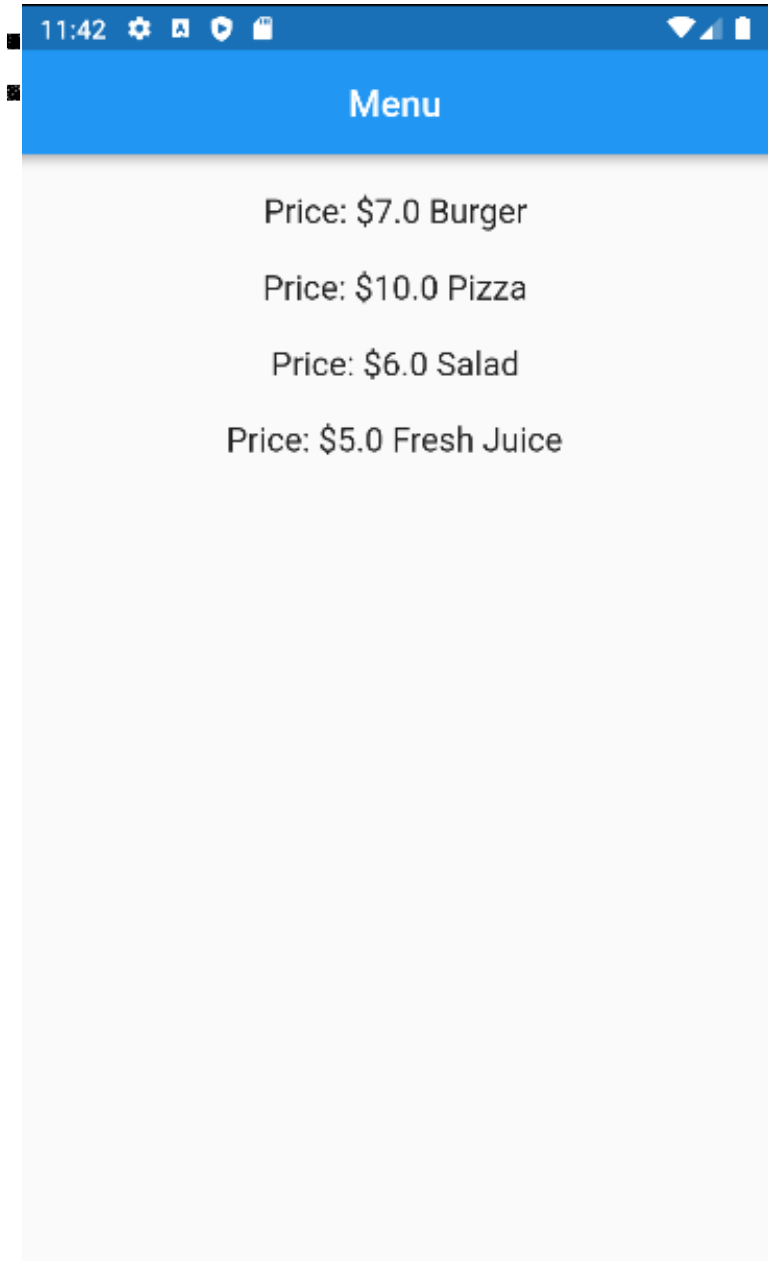


EXAMPLE LISTVIEWS:



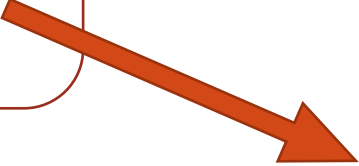
WEEK 8 APPLICATION PART 1:

- We will start by creating an application that only displays text items in a ListView.



WE WILL FIRST CREATE A CLASS TO HOLD LISTVIEW ITEMS. IT WILL BE CREATED IN THE ITEM.DART FILE.

The toString() method is what will be displayed for each ListView item.



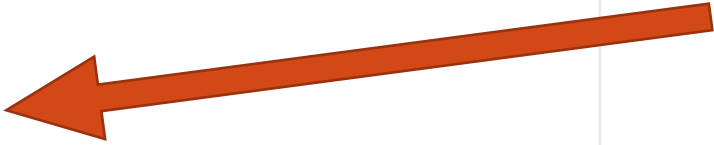
```
class Item {  
  final String _name;  
  final double _price;  
  
  Item(this._name, this._price);  
  
  // getters and setters  
  String get name => _name;  
  double get price => _price;  
  
  // item description is displayed in the ListView  
  @override  
  String toString() {  
    String space = ''; // loop computes spaces between price and name  
    for (int i = 0; i < 3 - _price.toString().length; i++) {  
      space += ' ';  
    }  
    return 'Price: \$_price $space$_name';  
  }  
}
```

WE WILL THEN CREATE A LIST OF ITEM OBJECTS.

```
// list of items, used to populate ListView  
List<Item> items = [  
    Item('Burger', 7),  
    Item('Pizza', 10),  
    Item('Salad', 6),  
    Item('Fresh Juice', 5),  
];
```

CREATING THE SHOWITEMS WIDGET

```
class ShowItems extends StatelessWidget {  
  const ShowItems({required this.width, Key? key}) : super(key: key);  
  final double width;  
  
  @override  
  Widget build(BuildContext context) {  
  
    return ListView.builder(  
      padding: const EdgeInsets.all(10),  
      itemCount: items.length,  
      itemBuilder: (context, index) {  
        return Column(children: [  
          const SizedBox(height: 10),  
          SizedBox(width: width * 0.28),  
          Text(items[index].toString(), style: const TextStyle(fontSize: 18)),  
          const SizedBox(height: 10)  
        ]); // Column  
      },  
    ); // ListView.builder  
  }  
}
```



The ListView will be populated from the Items list.

NEXT WE WILL ADD THE
SHOWITEMS WIDGET TO THE
HOME.DART FILE.

WE USED THE MEDIAQUERY
WIDGET TO DETERMINE THE
WIDTH ACCORDING TO THE
SCREEN SIZE.

```
import 'package:flutter/material.dart';
import 'item.dart';

class Home extends StatefulWidget {
  const Home({super.key});

  @override
  State<Home> createState() => _HomeState();
}

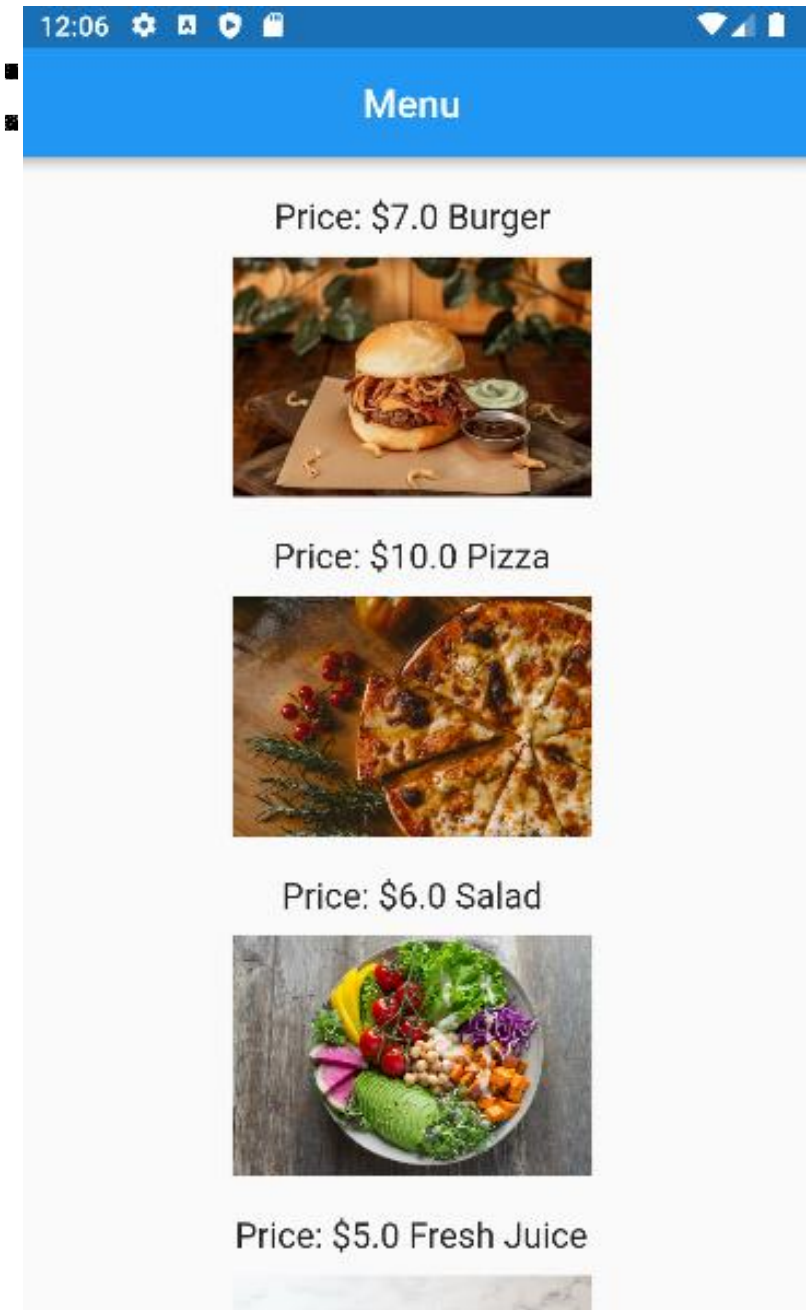
class _HomeState extends State<Home> {

  @override
  Widget build(BuildContext context) {
    double screenWidth = MediaQuery.of(context).size.width;
    if (MediaQuery.of(context).orientation == Orientation.landscape) {
      screenWidth = screenWidth * 0.8;
    }

    return Scaffold(
      appBar: AppBar(
        // displays the total price
        title: const Text('Menu'),
        centerTitle: true,
      ), // AppBar
      // check if we need to display selected items or menu
      // it depends on the _showSelected field
      body: ShowItems(width: screenWidth)
    ); // Scaffold
  }
}
```

WEEK 8 APPLICATION PART 2:

- We will modify the application to add an image to each item in the list.



NOW WE WILL MODIFY THE ITEM CLASS TO ADD AN IMAGE FIELD, THAT HOLDS IMAGE URLS THAT WE WILL RETRIEVE FROM THE NET.

```
class Item {  
    final String _name;  
    final double _price;  
    final String _image; // holds image url  
  
    Item(this._name, this._price, this._image);  
  
    // getters and setters  
    String get name => _name;  
    double get price => _price;  
    String get image => _image;  
  
    // item description is displayed in the ListView  
    @override  
    String toString() {  
        String space = ''; // loop computes spaces between price and name  
        for (int i = 0; i < 3 - _price.toString().length; i++) {  
            space += ' ';  
        }  
        return 'Price: \$_price $space$_name';  
    }  
}
```

WE WILL MODIFY THE ITEMS LIST, TO ADD THE URL'S OF IMAGES. THE COMPLETE URL'S ARE TOO LARGE TO SHOW HERE. YOU CAN FIND THEM INSIDE THE APPLICATION ON GITHUB PART2 BRANCH.

```
// list of items, used to populate ListView
List<Item> items = [
    Item('Burger', 7,
        "https://plus.unsplash.com/premium_photo-1667682209935-b6c87cced668?w=400&auto=format&fit=crop&q=60"),
    Item('Pizza', 10,
        "https://images.unsplash.com/photo-1513104890138-7c749659a591?q=80&w=1470&auto=format&fit=crop&ixlib=photos-1.3.0&fm=jpg&crop=entropy&cs=tinysrgb&h=1080&w=1470&auto=format&fit=crop&q=60"),
    Item('Salad', 6,
        "https://images.unsplash.com/photo-1512621776951-a57141f2eefd?q=80&w=1470&auto=format&fit=crop&ixlib=photos-1.3.0&fm=jpg&crop=entropy&cs=tinysrgb&h=1080&w=1470&auto=format&fit=crop&q=60"),
    Item('Fresh Juice', 5,
        "https://images.unsplash.com/photo-1579954115545-a95591f28bfc?q=80&w=1470&auto=format&fit=crop&ixlib=photos-1.3.0&fm=jpg&crop=entropy&cs=tinysrgb&h=1080&w=1470&auto=format&fit=crop&q=60"),
];
```

DISPLAYING IMAGES USING URL:

- ❑ You can use the `Image.network` constructor, to retrieve an image asynchronously from a remote server.
- ❑ Below is an example:

```
Image.network('https://picsum.photos/250?image=9'),
```



MODIFY THE SHOWITEMS WIDGET

```
class ShowItems extends StatelessWidget {  
  const ShowItems({required this.width, Key? key}) : super(key: key);  
  final double width;  
  
  @override  
  Widget build(BuildContext context) {  
  
    return ListView.builder(  
      padding: const EdgeInsets.all(10),  
      itemCount: items.length,  
      itemBuilder: (context, index) {  
        return Column(children: [  
          const SizedBox(height: 10),  
          SizedBox(width: width * 0.28),  
          Text(items[index].toString(), style: const TextStyle(fontSize: 18)),  
          const SizedBox(height: 10),  
          // get image from url stored in Item image field  
          Image.network(items[index].image,  
            height: width * 0.3), // Image.network  
          const SizedBox(height: 10),  
        ]); // Column  
      },  
    ); // ListView.builder  
  }  
}
```